



Olei API Milestones 0-4 Complete quiz and answer guide

A practical review of repository assessment, container-ready Node.js lifecycle, production Docker images, AWS identity and cost safety, and Amazon ECR publication.

Includes: 106 questions with model answers, real troubleshooting lessons, Docker and AWS concept maps, interview scenarios, and a readiness checklist.



How to use this guide

Recommended study method

Cover each answer, explain the concept aloud in your own words, then compare. For interview practice, keep the first answer under 60 seconds and add technical detail only when asked.

Pass standard: You should be able to answer the core questions without memorizing commands. Commands are evidence of practice; the goal is to explain what changes, what does not change, where state lives, what can fail, and how you would verify or clean up.

Contents

- Milestone 0 - Repository audit and deployment scope
- Milestone 1 - Container-ready Node.js lifecycle
- Milestone 2 - Docker production image
- Milestone 3 - AWS identity, Region, and cost safety
- Milestone 4 - Amazon ECR publication and scanning
- Docker master review
- AWS master review
- Troubleshooting scenarios
- Interview round and final answer framework

Milestone 0 - Repository audit and deployment scope

Goal: understand the existing application, its ownership boundaries, dependencies, state, and deployment risks before changing it.

1. What is the difference between development, staging, and production?

Answer: Development is where engineers build and debug. Staging is a production-like environment used to validate changes safely. Production serves real users and data. Each environment should have separate configuration, credentials, access controls, and operational approval.

2. Why should the same immutable image be used in staging and production with different injected configuration?

Answer: Reusing the exact image proves that production runs the artifact already tested in staging. Rebuilding can introduce different dependencies or output. Environment-specific hosts, credentials, and feature settings should be injected at runtime rather than baked into the image.

3. What is a deployment artifact?

Answer: A deployment artifact is the versioned output that is promoted and executed, such as a Docker image. It should be reproducible, immutable, and independent from environment-specific secrets.

4. What is the difference between application code and infrastructure?

Answer: Application code implements business behavior. Infrastructure supplies the runtime environment: networks, compute, registries, IAM, secret storage, logging, and routing. They interact, but ownership and rollback may differ.

5. Which Olei dependencies are stateful?

Answer: SQL Server stores durable business data. Redis stores sessions and other shared temporary state. They must survive replacement of an API process or container.

6. Which process-local state can safely disappear when a task stops?

Answer: SQL connection pools, the pool cache, timers, compiled files inside the image, temporary export files, and normal process memory can disappear if a replacement task can recreate them.

7. Why are SQL connection pools not durable application state?

Answer: A pool is a process-local performance mechanism containing open connections. It can be rebuilt after restart. The database records reached through the pool are durable; the pool itself is disposable.

8. Why does infrastructure ownership matter when the backend owner does not own the database?

Answer: Permission to deploy code does not grant permission to change database firewalls, credentials, TLS policy, data scope, or connection limits. The database owner must authorize those changes.

9. What is the difference between configuration and secrets?

Answer: Configuration controls behavior, such as ports or Region. Secrets grant access, such as passwords, tokens, and private keys. Some configuration values can still be operationally sensitive, but secrets require stronger storage, access control, rotation, and redaction.

10. What authorization was required before uploading the proprietary repository as an image?

Answer: Authorization from the responsible Olei backend owner to package and deploy the code, plus an approved staging environment. Database ownership remained separate, so deployment authorization did not authorize database or firewall changes.

11. Why must a tracked SMTP credential be treated as compromised?

Answer: Anyone with repository access or historical copies may have seen it. Removing it from the current file does not erase history. It must be rotated or revoked, logs reviewed, and history-remediation scope decided.

12. What makes this legitimate professional experience rather than an invented tutorial?

Answer: It uses an authorized existing application, preserves its architecture, records real constraints, implements and verifies operational behavior, respects ownership boundaries, and describes only work actually completed.

Milestone 0 interview summary

I audited an existing Node.js backend before deployment, separated durable dependencies from disposable process state, inventoried configuration without exposing values, documented ownership boundaries, and selected an authorized staging path.

Milestone 1 - Container-ready Node.js lifecycle

Goal: make the API start, report health, stop, and restart safely inside an orchestrated container.

1. What does `server.listen()` do?

Answer: It binds the HTTP server to a network port and begins accepting connections. Creating an Express application does not by itself make it reachable; `listen` opens the network listener.

2. Why should the API connect to required dependencies before calling `listen`?

Answer: Otherwise it may accept requests while SQL Server or Redis is unavailable. Ordered startup prevents false availability and makes startup failure explicit.

3. What does Redis lazy connection mean?

Answer: The Redis client object can be created without immediately opening a network connection. Startup code explicitly calls `connect` at the correct time, making ordering, errors, and cleanup predictable.

4. What was the risk when Redis was not closed after startup failure?

Answer: A partially initialized Redis connection could remain open and keep resources or the Node.js event loop alive. Startup rollback must close every dependency that opened successfully before the later failure.

5. Why can `createServer` be `async` even if `new Server(port)` itself is synchronous?

Answer: The asynchronous boundary allows runtime configuration validation and dynamic module loading to occur after validation. It does not mean the constructor automatically waits for HTTP, SQL, or Redis unless that logic is explicitly awaited elsewhere.

6. What is the difference between an `async` function returning `Promise` and a synchronous function returning `ApplicationServer`?

Answer: The `async` caller must await the `Promise` before receiving the server. A synchronous function returns the object immediately. `Async` is useful when preparation includes asynchronous work or when import timing must be controlled.

7. Why validate configuration before importing or constructing infrastructure modules?

Answer: Modules can have side effects such as creating clients or reading environment variables. Validating first guarantees missing or invalid configuration fails before dependency access and without partially starting the application.

8. Why is `process.env.DB_PASSWORD!` insufficient?

Answer: The TypeScript non-null assertion only silences the compiler. It performs no runtime check, so the value can still be undefined, empty, or malformed when the process starts.

9. If SQL Server becomes unavailable, which health endpoint should fail?

Answer: Readiness should fail because the task cannot safely serve normal traffic. Liveness should remain successful while the process itself is running, preventing an outage from causing pointless restart loops.

10. What is the difference between liveness and readiness?

Answer: Liveness answers whether the process is alive and should be restarted. Readiness answers whether it should receive traffic. A process can be live but temporarily unready.

11. What shutdown order should follow ECS SIGTERM?

Answer: Mark the service unready, stop accepting new connections, drain in-flight HTTP requests, stop timers or workers, close SQL pools and Redis, then exit. Cleanup should be bounded by a deadline.

12. Why does graceful shutdown need a deadline?

Answer: A dependency can hang forever. An orchestrator needs the process to terminate within its stop window. If cleanup exceeds the deadline, force termination and use a non-zero exit code to signal incomplete shutdown.

13. What exit code should represent successful and incomplete shutdown?

Answer: Exit code 0 represents a clean intentional shutdown. A failed or timed-out cleanup should use a non-zero code such as 1.

14. Why stop accepting requests before closing dependencies?

Answer: If dependencies close first while new requests are still accepted, those requests fail unpredictably. Closing the listener establishes a clean boundary and lets existing work finish against still-available dependencies.

15. Why write structured logs to stdout and stderr?

Answer: Containers are ephemeral, so logs should not depend on local files. Structured JSON can be collected by the platform, searched by fields, and redacted consistently.

Milestone 1 interview summary

I made the API fail fast on invalid configuration, start dependencies before accepting traffic, expose separate liveness and readiness endpoints, and implement bounded graceful shutdown for HTTP, SQL Server, Redis, and timers.

Milestone 2 - Docker production image

Goal: produce a reproducible, restricted, non-root Node.js runtime image and verify it locally.

1. What is the difference between a Docker image and a container?

Answer: An image is an immutable packaged filesystem plus metadata. A container is a running or stopped instance created from an image with a writable layer, process, network namespace, and runtime configuration.

2. What is a Dockerfile?

Answer: A Dockerfile is the ordered build recipe Docker uses to create image layers, copy files, install dependencies, set metadata, and define the default runtime process.

3. What is the Docker build context?

Answer: It is the directory tree sent to the Docker builder. COPY can only access files in that context. A large or sensitive context increases build time and accidental inclusion risk.

4. Why must .env be in .dockerignore even if the Dockerfile does not intentionally copy it?

Answer: Excluding it prevents the secret-bearing file from being sent to the builder and reduces the chance that a future broad COPY or build tool includes it.

5. What problem do multi-stage builds solve?

Answer: They allow compilers, TypeScript, tests, and development tooling in a build stage while copying only compiled output and production dependencies into the final runtime stage.

6. Why should TypeScript, Jest, ESLint, and other development tools be excluded from the runtime stage?

Answer: They are not required to run compiled JavaScript. Excluding them reduces image size, attack surface, download time, and unnecessary vulnerability findings.

7. What is an image layer?

Answer: A layer is a content-addressed filesystem change produced by a build instruction. Layers can be cached and shared across images when their content is identical.

8. Why copy package manifests before application source?

Answer: Dependency installation can remain cached when source code changes but package manifests do not. Copying all source first would invalidate the expensive dependency layer more often.

9. What is the difference between EXPOSE 5001 and docker run -p 8080:5001?

Answer: EXPOSE documents the container port expected by the application. It does not publish it. -p maps host port 8080 to container port 5001 so the host can reach it.

10. Why use CMD ["node", "dist/index.js"] instead of CMD node dist/index.js?

Answer: Exec form starts Node directly as PID 1, so it receives SIGTERM from Docker or ECS. Shell form adds a shell process that can complicate signal delivery and graceful shutdown.

11. Why run the application as a non-root user?

Answer: If the application is compromised, the attacker has fewer permissions inside the container. Files and writable directories must be owned or accessible by that user.

12. Why is a container not a virtual machine?

Answer: Containers isolate processes and filesystems while sharing the host kernel. A VM runs a complete guest operating system with its own kernel.

13. What caused "open Dockerfile: no such file or directory"?

Answer: docker build used the current directory as its context, but the command was run from the home directory instead of the repository containing Dockerfile.

14. What caused Docker Desktop meta.db to report a read-only filesystem?

Answer: Docker Desktop internal storage became unhealthy or read-only. The repository and Dockerfile were present; restarting or repairing Docker Desktop addressed the daemon storage problem.

15. Why measure image size instead of simply calling the image small?

Answer: Image size is an empirical property. Measuring layers identifies where space is used and supports informed optimization rather than unsupported claims.

Milestone 2 interview summary

I created a Node.js 22 multi-stage Docker image, restricted the build context, pruned development dependencies, ran as a non-root user, and verified startup failure, health, writable temporary storage, and SIGTERM shutdown locally.

Milestone 3 - AWS identity, Region, and cost safety

Goal: establish safe human access, account recovery, naming, tags, and spending visibility before application infrastructure.

1. Why should the AWS root user not be used for daily work?

Answer: Root has unrestricted account authority, including sensitive account and billing operations. Daily work should use an IAM Identity Center role with MFA and only the permissions needed. Root should have MFA, no access keys, and be reserved for root-only tasks.

2. What is authentication versus authorization?

Answer: Authentication proves who a principal is. Authorization evaluates policies to determine which actions that principal may perform on which resources and under what conditions.

3. What is an AWS Region versus an Availability Zone?

Answer: A Region is a separate geographic AWS area. An Availability Zone is an isolated infrastructure location inside a Region. A Region contains multiple AZs.

4. Why can IAM Identity Center be in us-east-1 while workloads use us-east-2?

Answer: Identity Center configuration has its own home Region. Workload resources can be created in another supported Region; identity location does not force application resource location.

5. What is an IAM principal?

Answer: A principal is an identity that can make an AWS request, such as a user, assumed role, AWS service, or federated identity.

6. What are IAM users, roles, and policies?

Answer: A user is a long-lived IAM identity. A role is assumed to obtain temporary permissions. A policy is a JSON authorization document defining allowed or denied actions, resources, and optional conditions.

7. Why are temporary SSO credentials preferable to permanent access keys?

Answer: They expire automatically, are obtained after interactive authentication and MFA, and reduce the risk and rotation burden of long-lived secrets.

8. What does an AWS CLI profile do?

Answer: It names a configuration set such as SSO session, account role, default Region, and output format. Explicit profiles reduce the risk of running a command against the wrong identity.

9. Why verify account and Region before creating a resource?

Answer: Many AWS mistakes are valid commands executed in the wrong account or Region. Preflight verification prevents misplaced resources, unexpected access, and hidden costs.

10. Why use resource names and tags?

Answer: Names help humans identify purpose. Tags add consistent metadata for ownership, environment, cost allocation, inventory, and automation. Tags are not access controls or spending limits.

11. Does reaching 100% of an AWS Budget guarantee spending stops?

Answer: No. Budgets primarily monitor and alert, cost data can be delayed, and resources continue running unless separately controlled. No automatic budget actions were configured here.

12. What is the AWS shared responsibility model?

Answer: AWS secures the cloud infrastructure. The customer remains responsible for configuration, identities, data, application security, network rules, secrets, guest or container software, and service-specific controls.

13. Why were alternate billing, operations, and security contacts configured?

Answer: They route important AWS notices to controlled channels and reduce dependency on the root email for operational communication.

14. Why is AdministratorAccess acceptable only as a bootstrap state?

Answer: It enables initial setup but violates least privilege for routine deployment. Narrow deployment, task execution, and application roles should replace broad daily use.

15. Why does a USD 5 learning budget affect architecture decisions?

Answer: Always-on resources such as NAT Gateways or load balancers can exceed that amount quickly. The budget requires estimating billing dimensions and cleanup before resource creation.

Milestone 3 interview summary

I established MFA-protected IAM Identity Center access, removed daily root usage, configured a named AWS CLI profile, selected workload and identity Regions deliberately, created cost alerts, and documented naming, tagging, and cleanup controls.

Milestone 4 - Amazon ECR publication and scanning

Goal: store a versioned Docker image privately in AWS and prove its identity, retention, and scan behavior.

1. What is a container registry?

Answer: A registry stores and distributes container manifests, image indexes, configuration objects, and content-addressed layers. Docker clients push to and pull from it.

2. What is an ECR repository versus an image?

Answer: The repository is the named registry collection and policy boundary. Images are versioned artifacts stored inside it and referenced by tags or digests.

3. What is the difference between aws sso login and Docker login to ECR?

Answer: SSO authenticates the human and gives AWS CLI temporary role credentials. Docker is a separate client; it receives a short-lived registry token obtained through the AWS CLI.

4. What is the difference between docker tag and docker push?

Answer: Tag adds another name pointing to an existing local image. Push uploads missing layers plus the manifest or index to the remote registry.

5. Where was the application image stored before and after the push?

Answer: Before the push it existed only in local Docker storage. Afterward it existed both locally and remotely in private ECR. Removing a local tag did not delete the remote image.

6. What is an image tag versus an image digest?

Answer: A tag is a human-readable reference such as a Git-derived release label. A digest is a SHA-256 content identity. The digest is the stronger deployment reference.

7. Why avoid relying only on latest?

Answer: latest is conventional, not special, and is normally movable. It does not identify which source revision is deployed and makes rollback and audit evidence ambiguous.

8. What does tag immutability protect?

Answer: Once a tag points to content, another push cannot replace it with a different image. New builds must use new tags, preserving traceability.

9. What is an OCI image index?

Answer: It is a top-level manifest that references one or more platform-specific image manifests and possibly related build metadata. The published tag pointed to an index containing the Linux ARM64 image.

10. Why did waiting for a scan by tag return `ScanNotFoundException`?

Answer: The tag resolved to the OCI index, but ECR Basic Scanning produced findings for the Linux ARM64 child image. Resolving the index and querying the child digest selected the correct scan target.

11. What does ECR Basic Scanning check?

Answer: It compares operating-system packages in the image with known vulnerability data. It does not test application logic, authorization, secrets, runtime reachability, or every language dependency.

12. What were the scan results?

Answer: The scan reported 3 critical, 5 high, 11 medium, and 1 low finding. All critical and high results mapped to the Debian Perl source package inherited through perl-base.

13. Why does a critical finding not automatically prove the Node.js API is exploitable?

Answer: Severity describes the vulnerability in its affected context. Exploitability also depends on whether the component and vulnerable behavior are reachable, attacker-controlled input, runtime permissions, and compensating controls.

14. Why was perl-base not removed immediately?

Answer: It is an essential Debian package. Removing it manually can break the runtime filesystem or package tooling. The supported remediation is a patched base or carefully tested alternative base, not blind package deletion.

15. What does an ECR lifecycle policy do?

Answer: It automatically expires images matching ordered retention rules. Here it limits untagged objects and keeps only a bounded number of Git-tagged images, controlling clutter and storage cost.

16. What remains billable after Docker logout?

Answer: The image layers retained in ECR remain billable storage. Docker logout removes local registry credentials only; it does not delete AWS resources.

17. Why did removing the ECR-qualified local tag not remove the original local image?

Answer: Both local tags referenced the same image ID. Removing one reference leaves image data as long as another tag or container still references it.

18. Is this image ready for production?

Answer: No. It is retained for controlled learning and staging, but critical and high inherited findings require remediation or explicit security risk acceptance before production.

Milestone 4 interview summary

I published a Git-versioned ARM64 image to private ECR, verified its digest and immutable tag policy, configured lifecycle retention, resolved an OCI index to its platform manifest for scanning, and investigated inherited Debian vulnerabilities without applying unsafe automatic fixes.

Docker master review

Use these questions to connect individual commands to the image lifecycle and operational model.

1. Explain the complete path from source code to a running container.

Answer: Source and dependency locks enter the build context; Docker executes the Dockerfile into cached layers; the final image receives a tag; Docker creates a container with runtime configuration and a writable layer; the configured process starts and listens on its container port.

2. Which values belong at build time and which belong at runtime?

Answer: Compilation inputs and locked dependencies belong at build time. Environment-specific hosts, ports, credentials, secrets, and operational toggles belong at runtime. A single immutable image should move between environments.

3. What makes a Docker build reproducible?

Answer: A pinned runtime family or digest, committed lockfile, deterministic install command such as `npm ci`, controlled build context, explicit copied files, and consistent platform improve reproducibility.

4. Why is a writable container filesystem not suitable for durable business data?

Answer: The container can be replaced at any time and its writable layer disappears. Durable state must live in external systems or explicitly managed persistent storage.

5. How do PID 1 and signals affect Node.js containers?

Answer: The main container process is PID 1 and receives stop signals. Exec-form CMD makes Node the main process so its SIGTERM handler can drain and close resources.

6. Why must ARM64 versus AMD64 be explicit?

Answer: Images contain platform-specific binaries. The Fargate runtime architecture must match the image manifest, or task startup fails. A multi-platform index can reference variants for both architectures if both are built.

7. What information is safe to place in image metadata and tags?

Answer: Non-secret operational metadata such as application name, source revision, version, and platform is appropriate. Credentials, tokens, private endpoints, customer data, and personal information are not.

8. How would you verify a production container image?

Answer: Inspect user, command, platform, ports, and contents; run missing-config failure tests; start against approved staging dependencies; check liveness/readiness; deliver SIGTERM; confirm clean exit; scan the platform manifest; and record the digest.

AWS master review

These answers focus on the AWS services actually used so far and the controls needed before ECS.

1. Which AWS resources currently exist because of this project?

Answer: Identity and governance controls, two budgets, an ECR repository, and one retained ECR image exist. No ECS, Fargate, load balancer, NAT Gateway, application VPC resources, secret store, or CloudWatch application log group has been created.

2. Which current resource generates ongoing application-project cost?

Answer: Retained ECR image storage. The planning estimate for the single image is small, but it continues until the image and repository are deleted according to an authorized cleanup plan.

3. Why must profile and Region be explicit in AWS CLI commands?

Answer: They constrain the identity context and resource location. This reduces the chance of creating or inspecting resources in the wrong account or Region.

4. Why is ECR authentication short-lived?

Answer: The Docker authorization token is derived from temporary AWS authorization and expires, reducing exposure compared with a permanent registry password.

5. What is least privilege in this project?

Answer: Humans and workloads should receive only the actions required on specific resources. Future ECS execution and application roles must be separate and narrower than the bootstrap administrator permission.

6. Why do secrets and networking solve different problems?

Answer: Secret storage controls possession of credentials. Networking controls which sources and destinations can communicate. A database password in Secrets Manager does not make a publicly exposed SQL port safe.

7. Why are inbound API traffic and outbound database traffic separate designs?

Answer: Inbound design determines how users reach the API through DNS, TLS, load balancing, and security groups. Outbound design determines how tasks reach SQL Server and whether the database can restrict a stable source path.

8. What should be recorded before creating a cost-generating AWS resource?

Answer: Purpose, account and Region, billing dimensions, expected monthly cost, expected lifetime, security implications, tags, verification steps, and exact cleanup command or console procedure.

Troubleshooting scenarios

Answer each scenario by separating symptom, likely layer, evidence, safe next step, and cleanup.

1. docker build cannot find Dockerfile, but ls shows it in the repository. What do you check?

Answer: Check pwd and the final build-context argument. A dot means the current directory. Run from the repository root or provide the correct explicit context and Dockerfile path.

2. Docker reports an internal containerd metadata database as read-only. Is that an application bug?

Answer: Probably not. The failure is inside Docker Desktop storage before the Dockerfile is evaluated. Verify disk health and restart or repair Docker Desktop before changing application code.

3. AWS CLI says the SSO profile lacks sso_start_url and sso_region. What happened?

Answer: The profile is incomplete or the wrong profile variable was used. Configure it with aws configure sso and then authenticate with the exact named profile.

4. The AWS access portal shows zero accounts after creating a user. What is missing?

Answer: Identity Center needs an account assignment connecting the user or group to a permission set for the AWS account. Creating the identity alone does not authorize account access.

5. ECR push succeeds but scan lookup by tag fails. What do you inspect?

Answer: Inspect the tag media type. If it is an OCI index, resolve the platform-specific child manifest and query scan findings using that child digest.

6. A dependency outage makes readiness fail while liveness stays healthy. Is that correct?

Answer: Yes, if the process is alive but cannot safely serve requests. Traffic should stop while the orchestrator avoids unnecessary restarts caused by an external outage.

7. SIGTERM arrives while requests are active. What should happen?

Answer: Mark unready, stop accepting new requests, allow in-flight requests to drain within a deadline, close dependencies and timers, then exit 0 if complete or non-zero if cleanup fails or times out.

8. A vulnerability scanner reports critical CVEs in an inherited OS package. What should you do?

Answer: Identify the exact package and platform manifest, check vendor status and fixed versions, assess reachability, rebuild from a patched supported base when available, rescan, and document any temporary risk decision. Do not blindly remove essential packages.

Interview round - model answers

These questions combine concepts across milestones. The answers show a strong structure, not a script you must memorize.

1. Tell me about the containerization work you completed.

Answer: I audited an existing TypeScript API, corrected its lifecycle for container orchestration, built a non-root multi-stage Node.js image, verified health and SIGTERM behavior against approved staging services, and published a Git-versioned ARM64 artifact to private ECR with immutable tags, lifecycle retention, digest verification, and scanning.

2. How did you prevent environment-specific configuration from entering the image?

Answer: The build context excludes .env and other local artifacts. The application validates required environment variables at startup, and environment-specific values are injected only when the container runs.

3. How would ECS determine whether your API should receive traffic?

Answer: It should use readiness for traffic eligibility and liveness for process health. Readiness becomes false during startup, dependency loss, and shutdown; liveness remains true while the process is functioning.

4. How did you make releases traceable?

Answer: The ECR image uses a Git-derived immutable tag, and the remote digest was verified. Future deployments should record and use the digest so the exact tested content is unambiguous.

5. What security issue remains?

Answer: The current Debian Bookworm runtime contains perl-base findings reported as critical and high. The application has no explicit Perl launch path, but that does not prove non-exploitability, so production remains blocked pending a patched base or documented security acceptance.

6. What would you improve next?

Answer: Design network paths and cost trade-offs before creating compute, then create least-privilege ECS roles and secret injection, deploy the matching ARM64 image to Fargate, add CloudWatch logs, and prove secure SQL connectivity and rollback.

7. Can you claim production AWS deployment experience now?

Answer: No. The completed work supports honest claims about Docker, AWS identity setup, and ECR publication, but the CV deployment gate requires a running ECS/Fargate service, secure secrets, logs, database connectivity, failure diagnosis, and rollback.

Final readiness checklist

Capability	Current status
Repository and deployment-risk audit	Completed
Container-safe Node.js lifecycle	Completed and tested
Production Docker image	Completed and tested locally

Capability	Current status
Safe AWS human identity and cost alerts	Completed
Private ECR publication and digest verification	Completed
Critical/high image findings	Open - production blocker
AWS networking design	Not started
Least-privilege ECS roles and secrets	Not started
Running ECS/Fargate service	Not started
CloudWatch operational diagnosis	Not started
Secure SQL Server connectivity from AWS	Not started
Deployment rollback proof	Not started

CV statement

Not ready yet for an end-to-end AWS deployment claim. You can honestly discuss Docker, container lifecycle, AWS CLI and IAM Identity Center setup, Amazon ECR, immutable image versioning, digest verification, lifecycle policies, and basic vulnerability scanning as completed hands-on work.